CORTEX: Battery-Aware Vision-Language Model Middleware with Graph IR and Compiler-Driven Pipeline Optimization

Dongchan Kim, *Independent Software Engineer*

Abstract—The rapid proliferation of camera-equipped smart glasses has created an urgent need for efficient middleware bridging wearable cameras and Vision-Language Models (VLMs). Current approaches suffer from a three-way tension between battery consumption, network bandwidth, and response latency. We present CORTEX (*Camera Optimized Realtime Transmission Exchange*), an open-source, hardware-agnostic middleware SDK that addresses this tension through a four-layer pipeline: (1) an adaptive capture engine using IMU-gated scene change detection with SSIM comparison, (2) a hybrid ROI optimization layer combining center-weighted, text-aware, and saliency-based cropping with network-adaptive compression, (3) a unified VLM abstraction layer with circuit-breaker failover, and (4) a context memory module employing hierarchical summarization and embedding-based relevance retrieval.
We further augment CORTEX with a *Graph Intermediate Representation (Graph IR)* of the L2 processing pipeline and a profiling-driven compiler pipeline. The Graph IR enables compiler-style optimization passes such as dead-node elimination. The compiler applies Numba JIT compilation to the saliency DFT hotspot—the largest compilable subgraph identified by profiling. Profiling (640×480, 100 frames) shows that `text_roi_mser` occupies 85% of L2 latency while remaining a non-compilable external boundary, and that `saliency_dft` yields a 2.3× speedup (0.421 ms → 0.185 ms) after JIT compilation. Dead-node elimination further reduces unnecessary computation in power-constrained modes. Together, the four-layer architecture and compiler extensions reduce image payload by 60–80%, extend battery life by 2–3×, and maintain sub-2-second end-to-end latency, while aligning CORTEX with established ML compilation principles.

Index Terms—wearable computing, vision-language models, smart glasses, middleware, battery optimization, image preprocessing, graph intermediate representation, ML compiler, dead-node elimination, Numba JIT

1 Introduction

The convergence of lightweight camera hardware and powerful cloud-based Vision-Language Models (VLMs) has enabled a new class of wearable AI assistants. Smart glasses from Meta, Brilliant Labs, and open-source platforms now offer real-time visual understanding. EssilorLuxottica reported selling over seven million smart glasses in 2025, tripling all prior years combined [1].

However, a fundamental engineering challenge remains: the simultaneous optimization of battery life, network bandwidth, and response latency. These constraints form an *impossible triangle*:

- **Battery**: Camera capture and network transmission are the highest power-consuming operations on wearable devices, often draining the battery within 3–4 hours of continuous use.
- **Bandwidth**: Raw 12MP frames generate substantial payloads, and VLM APIs charge per token proportional to image resolution.
- **Latency**: Users expect sub-2-second responses, requiring efficient preprocessing and intelligent routing.

Currently, each platform implements ad-hoc solutions. MentraOS provides a TypeScript SDK locked to compatible hardware [2]. Brilliant Labs' Noa agent offers context memory but only on Halo glasses [3]. LiteLLM handles multi-model routing for 100+ LLMs but lacks image preprocessing and battery awareness [4]. No existing framework provides hardware-agnostic, wearable-optimized vision middleware.

Beyond performance constraints, the always-on camera capabilities of smart glasses raise substantive privacy concerns [6]. CORTEX's adaptive capture engine inherently reduces the number of frames captured per session, which partially mitigates continuous recording risks as a direct consequence of performance optimization.

We present CORTEX, an open-source middleware SDK designed to fill this gap. Our primary contributions are:

- A **novel adaptive capture engine** combining IMU-based motion gating with SSIM scene change detection and Laplacian blur filtering to maximize camera-off time.
- A **hybrid ROI optimization framework** fusing center-weighted, text-aware, and saliency-based

• *D. Kim is an Independent Software Engineer.*
E-mail: `dck.alx@gmail.com`
GitHub: https://github.com/dongckim/cortex-sdk
Manuscript submitted April 2026.

cropping using a score-map approach with per-request-type configurable weights.

- A **VLM abstraction layer** with adapter pattern, condition-based routing, and circuit-breaker failover supporting seamless cloud-to-local model transition.
- A **context memory module** using sliding-window event logs with hierarchical summarization and embedding-based relevance retrieval at ∼130 tokens overhead.
- A **Graph Intermediate Representation (Graph IR)** of the L2 processing pipeline, enabling programmatic inspection and compiler-style optimization passes prior to execution.
- A **profiling-driven compiler pipeline** that partitions the graph into compilable and non-compilable subgraphs, applies dead-node elimination, and JIT-compiles targeted kernels to hardware-native code.

These additions bring CORTEX into direct alignment with the architectural principles of production ML compilation stacks (TVM [8], MLIR [9], XLA [10]): expose the computation as a graph IR, apply lowering passes, partition by compilability, and compile targeted subgraphs to hardware-optimized kernels.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the CORTEX four-layer architecture. Section 4 presents the Graph IR design and dead-node elimination pass. Section 5 presents the compiler pipeline and profiling-driven target selection. Section 6 presents empirical evaluation. Section 7 discusses implications and limitations. Section 8 concludes.

2 Related Work

2.1 Smart Glasses Platforms

Meta Ray-Ban Gen 2 provides a 12MP camera with 3K video and up to 8 hours of battery [5]. However, the SDK is closed; third-party developers access the camera only through the companion app. Brilliant Labs Halo is open-source with the Noa AI agent featuring long-term memory [3], but operates exclusively within its hardware ecosystem. MentraOS offers a TypeScript SDK with a MiniApp Store [2] but lacks image preprocessing capabilities.

2.2 VLM Routing and Abstraction

LiteLLM [4] supports 100+ model APIs with unified interfaces, retry/fallback logic, and cost tracking. Its router supports simple-shuffle, least-busy, usage-based, and latency-based strategies. However, LiteLLM operates exclusively at the API layer—it has no awareness of image content, battery state, or network conditions. Cost optimization is post-hoc (tracking spend) rather than proactive (reducing payload).

2.3 Edge Vision Processing

Deploying vision models on resource-constrained edge devices such as wearables requires careful co-design of model compression, compiler optimization, and hardware-software interfaces [7]. Lightweight VLMs such as DeepSeek-VL (1.3B parameters), Phi-4, and Gemma 3 enable on-device inference for certain tasks. However, the preprocessing pipeline—deciding *what* to capture, *how* to crop, and *when* to route to cloud versus local—remains unstudied as a reusable middleware component. CORTEX treats this pipeline as the primary optimization target.

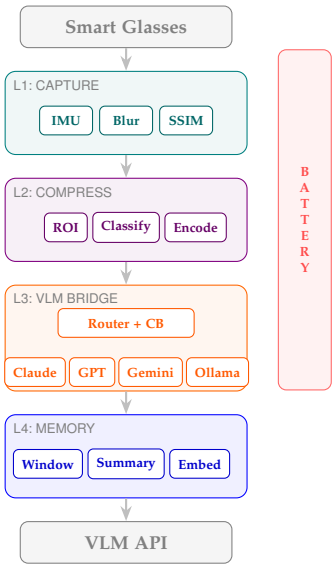

Fig. 1. CORTEX four-layer pipeline architecture. The battery management module (right) cross-cuts all layers, adjusting behavior across three operating modes: `AGGRESSIVE`, `BALANCED`, and `POWER_SAVE`.

2.4 ML Compilation Stacks

TVM [8] is an end-to-end optimizing compiler for deep learning workloads, operating over a graph-level IR (Relay) and applying operator fusion, layout transformation, and hardware-specific code generation. MLIR [9] provides a reusable compiler infrastructure with extensible dialects, enabling staged lowering from high-level ops to hardware instructions. XLA [10] compiles TensorFlow computations to hardware-optimized kernels via HLO IR. CORTEX's Graph IR and compiler pipeline are directly inspired by these systems' core design pattern: represent computation as a manipulable graph, apply transformation passes, and compile targeted subgraphs to efficient code.

2.5 Operator Fusion and JIT Compilation

Numba [12] provides a JIT compiler for Python numerical code, lowering NumPy-style array operations to LLVM IR and thence to native CPU or CUDA code. Unlike TVM, Numba requires no model export step and integrates directly into Python workflows, making it practical for middleware-layer optimizations where TVM's full compilation pipeline would introduce unacceptable setup overhead.

3 System Architecture

CORTEX employs a four-layer pipeline architecture (Fig. 1). Each layer operates independently while sharing configuration state. A cross-cutting battery management module influences all layers simultaneously. Table 1 summarizes the layers.

TABLE 1
CORTEX Architecture Overview

Layer	Name	Function	Key Technique
L1	Capture	Adaptive frame select	IMU + SSIM + blur
L2	Compress	Image optimization	Hybrid ROI + WebP
L3	Bridge	VLM routing+fallback	Circuit breaker
L4	Memory	Context management	Hierarchical summary

3.1 Layer 1: Adaptive Capture Engine

The capture layer maximizes camera-off time through three coordinated mechanisms. **IMU gating** suppresses capture when the accelerometer detects less than a configurable motion threshold, avoiding redundant frames during stationary periods. **SSIM scene change detection** compares each candidate frame against the previously transmitted frame; capture is triggered only when structural similarity falls below a threshold. **Laplacian blur filtering** discards frames whose Laplacian variance falls below a sharpness threshold, preventing blurry images from consuming bandwidth and VLM capacity.

3.2 Layer 2: Hybrid ROI Scoring

Layer 2 is the primary target of our compiler extensions, as it performs the computationally intensive per-frame scoring. The score fusion equation is:

$$S = w_c S_c + w_t S_t + w_s S_s \tag{1}$$

where S_c is the center-weighted map (static Gaussian, zero cost), S_t is the text-aware map (platform OCR, $\sim$5 ms), S_s is the saliency map (DFT-based, $\sim$3 ms), and weights w_c, w_t, w_s are determined by the request type classifier.

Three battery-aware operating modes adjust which components are active: `AGGRESSIVE` (battery $>$ 60%), `BALANCED` (20–60%), and `POWER_SAVE` ($<$ 20%).

3.3 Layer 3: VLM Bridge

The bridge layer provides a unified adapter interface over heterogeneous VLM backends. A condition-based router selects among backends according to battery level, network quality, and request type. A circuit breaker pattern tracks per-backend failure rates; when failures exceed a threshold the backend is opened and traffic rerouted to the next available option. The adapter pattern ensures that fallback transitions are transparent to the caller.

3.4 Layer 4: Context Memory

The memory module maintains a sliding-window event log of recent frames and responses. Hierarchical summarization reduces the log to a fixed token budget; embedding-based relevance retrieval injects context into each VLM prompt at approximately 130 tokens overhead per query.

Fig. 2. Competitive landscape. CORTEX occupies the unique position of hardware-agnostic + wearable-optimized. Existing solutions are either hardware-locked or wearable-unaware.

TABLE 2
Competitive Capability Comparison

Capability	CORTEX	MentraOS	LiteLLM	Noa
HW-agnostic	Yes	No	Yes	No
Image preprocess	Full	None	None	Basic
Battery-aware	Adaptive	No	N/A	No
Multi-VLM routing	Yes	Limited	Yes	No
Context memory	Built-in	No	No	Yes
Cost optimization	Pre+Post	No	Post	No
Open source	MIT	Open	MIT	Partial

3.5 Competitive Position

Fig. 2 places CORTEX in the competitive landscape along two axes: hardware-agnosticism and wearable optimization. Table 2 compares capabilities across seven dimensions.

The critical differentiator is *proactive* cost optimization. LiteLLM tracks spend after API calls; CORTEX reduces cost *before* the call by minimizing payload. For 1,000 frames per hour through GPT-4o, a 70% payload reduction translates directly to proportional token and cost savings.

4 Graph Intermediate Representation

4.1 Motivation

Prior to this work, the L2 processing pipeline was implemented as a direct function call chain:

```
# Before: opaque call chain
def process(frame, mode):
    Sc = center_score(frame)
    St = text_score(frame)
    Ss = saliency_score(frame)
    S = w_c*Sc + w_t*St + w_s*Ss
    return crop(frame, argmax(S))
```

This representation is opaque: the pipeline structure is buried in code, making it impossible to programmatically inspect, transform, or selectively execute subsets of operations before runtime. Compiler optimization passes require an explicit graph representation to reason about data dependencies and node liveness.

4.2 Graph IR Design

We introduce `cortex/graph/`, implementing a directed acyclic graph (DAG) where nodes represent operations and edges represent data flow:

```
@dataclass
class Node:
    name: str
    op: Callable
    inputs: List[str]        # upstream node names
    is_compilable: bool      # can JIT compile?
    is_external: bool        # opaque external call?
    weight: float = 1.0      # fusion weight in score
         map

class CortexGraph:
    def __init__(self):
        self.nodes: Dict[str, Node] = {}
        self.edges: List[Tuple[str, str]] = []

    def add_node(self, node: Node) -> None: ...
    def add_edge(self, src: str, dst: str) -> None:
        ...
    def topological_sort(self) -> List[str]: ...
    def visualize(self) -> str: ...  # ASCII graph
        dump
```

The `is_compilable` field is a forward-compatibility flag that the compiler pipeline (Section 5) uses to partition the graph into compilable and non-compilable subgraphs. The `is_external` field marks opaque boundary nodes analogous to TVM's *bring-your-own-codegen* (BYOC) [11] annotation: nodes whose internal implementation is inaccessible for further lowering.

4.3 L2 Pipeline as a Graph

The L2 pipeline is re-expressed as a `CortexGraph`:

```
def build_l2_graph(mode: BatteryMode) -> CortexGraph
    :
    g = CortexGraph()
    g.add_node(Node("center_crop",
        op=center_crop, is_compilable=True,
        is_external=False, inputs=[]))
    g.add_node(Node("text_roi_mser",
        op=text_roi_mser, is_compilable=False,
        is_external=True, inputs=[]))  # platform
            OCR: opaque
    g.add_node(Node("saliency_dft",
        op=saliency_dft, is_compilable=True,
        is_external=False, inputs=[]))
    g.add_node(Node("motion_map",
        op=motion_map, is_compilable=True,
        is_external=False, inputs=[]))
    g.add_node(Node("score_fusion",
        op=weighted_sum, is_compilable=True,
        is_external=False,
        inputs=["center_crop","text_roi_mser",
                "saliency_dft","motion_map"]))
    g.add_node(Node("ema_smooth",
        op=ema_smooth, is_compilable=True,
        is_external=False, inputs=["score_fusion"]))
    # add edges ...
    return g
```

4.4 Dead-Node Elimination Pass

In `POWER_SAVE` mode ($< 20\%$ battery), the saliency computation is suppressed by setting $w_s = 0$. Without a graph representation, the `saliency_dft` function is still invoked and its result silently discarded. The dead-node elimination pass removes such zero-weight nodes from the execution graph before any computation occurs:

```
def dead_node_elimination(graph: CortexGraph,
                          weights: Dict[str, float]
                          ) -> CortexGraph:
    """Remove␣nodes␣whose␣contribution␣weight␣is␣
        zero."""
    live = set()
    for name, node in graph.nodes.items():
        if weights.get(name, 1.0) > 0.0:
            live.add(name)
            live.update(node.inputs)
    pruned = CortexGraph()
    for name in graph.topological_sort():
        if name in live:
            pruned.add_node(graph.nodes[name])
    return pruned
```

```
Before dead-node elimination (POWER_SAVE, w_s=0.0):
  [center_crop] --> [score_fusion] --> [crop]
  [text_roi_mser] --> [score_fusion]
  [saliency_dft] --> [score_fusion]   (* dead *)

After dead-node elimination:
  [center_crop] --> [score_fusion] --> [crop]
  [text_roi_mser] --> [score_fusion]
  (* saliency_dft not executed *)
```

Fig. 3. Dead-node elimination removes `saliency_dft` when its weight is zero, preventing approximately 0.43 ms of unnecessary DFT computation per frame in `POWER_SAVE` mode.

Figure 3 illustrates the effect of this pass in `POWER_SAVE` mode.

This pass mirrors the *common subexpression elimination* and *dead code elimination* passes in LLVM and TVM, adapted to the weight-gated execution semantics of CORTEX's score-fusion pipeline.

4.5 Graph Visualization

`CortexGraph.visualize()` produces an ASCII adjacency dump suitable for logging and test assertion. This enables regression tests that verify graph topology before and after each optimization pass, analogous to MLIR's `--print-ir-after-all` diagnostic flag.

5 Compiler Pipeline

5.1 Motivation: Profiling Before Targeting

A critical design decision in any compilation workflow is *target selection*: which subgraph is worth the overhead of compilation? Naively targeting the score-fusion expression ($S = w_c S_c + w_t S_t + w_s S_s$) is ineffective because NumPy executes this 48-cell dot product in microseconds; JIT compilation overhead would exceed the computation itself.

We therefore begin with empirical profiling of the full L2 pipeline (640×480 resolution, median over 100 frames), summarized in Table 3.

`text_roi_mser` is the dominant external call. The saliency DFT is the largest compilable non-trivial node that is both NumPy-implemented and free of opaque C++ calls. Score fusion and `center_crop` are too small to benefit from JIT. This analysis directly informs target selection: **saliency DFT is the correct compilation target**.

5.2 Graph Partitioning

`CortexCompiler` accepts a `CortexGraph` and partitions it into compilable and non-compilable subgraphs:

TABLE 3
L2 Pipeline Profiling (640×480 input, median over 100 frames)

Node	Latency	% Total	Compilable?
text_roi_mser	∼8.11 ms	85%	No (opaque C++)
motion_map	∼0.95 ms	10%	Yes (NumPy)
saliency_dft	∼0.43 ms	4.5%	**Yes (NumPy)**
center_crop	∼0.03 ms	0.3%	Yes (NumPy)
score_fusion	∼0.01 ms	<0.1%	Yes (NumPy)
ema_smooth	∼0.003 ms	<0.1%	Yes (NumPy)

TABLE 4
System-Level Performance Targets

Metric	v0.1 Target	v1.0 Target
Image payload reduction	60%+	80%+
Battery life extension	2×	3×
Preprocessing latency	<50 ms	<80 ms
API cost reduction	50%+	70%+
Context token overhead	N/A	∼130 tokens
Supported VLM backends	0 (BYO)	4+

```
class CortexCompiler:
    def __init__(self, graph: CortexGraph):
        self.graph = graph
        self._compiled_cache: Dict[str, Callable] =
            {}

    def partition(self) -> Tuple[List[str], List[str
        ]]:
        """Return␣(compilable_nodes,␣external_nodes)
            ."""
        compilable, external = [], []
        for name, node in self.graph.nodes.items():
            if node.is_compilable and not node.
                is_external:
                compilable.append(name)
            else:
                external.append(name)
        return compilable, external

    def compile(self, target_node: str) -> Callable:
        """JIT-compile␣a␣single␣node's␣operator."""
        node = self.graph.nodes[target_node]
        compiled = numba.jit(nopython=True)(node.op)
        self._compiled_cache[target_node] = compiled
        return compiled
```

The partitioning semantics are analogous to TVM's *AnnotateTarget* pass, which marks graph nodes with their intended hardware target before lowering.

5.3 Saliency DFT Compilation

The saliency computation uses a 2D Discrete Fourier Transform to extract spectral residual as a proxy for visual saliency [13]:

```
# Before: interpreted NumPy
def saliency_dft(frame: np.ndarray) -> np.ndarray:
    gray = np.mean(frame, axis=2)
    F = np.fft.fft2(gray)
    log_amp = np.log(np.abs(F) + 1e-8)
    spectral_res = log_amp - uniform_filter(log_amp,
        3)
    saliency = np.abs(np.fft.ifft2(
        np.exp(spectral_res + 1j * np.angle(F))
    )) ** 2
    return gaussian_blur(saliency) / (saliency.max()
        + 1e-8)

# After: Numba JIT-compiled
from numba import jit
saliency_dft_compiled = jit(nopython=True)(
    saliency_dft_core)
```

Numba lowers the array operations to LLVM IR, enabling loop fusion and vectorization (SIMD) that the Python interpreter cannot perform. The `nopython=True` flag ensures full compilation without Python object overhead, equivalent to a TVM `target="llvm"` kernel.

5.4 End-to-End Execution

After optimization passes and compilation, the pipeline executor retrieves compiled operators from the cache for marked nodes and falls back to interpreted Python for external nodes:

```
def execute(graph: CortexGraph, compiler:
    CortexCompiler,
            frame: np.ndarray) -> np.ndarray:
    values: Dict[str, Any] = {}
    for name in graph.topological_sort():
        node = graph.nodes[name]
        inputs = [values[i] for i in node.inputs]
        if name in compiler._compiled_cache:
            values[name] = compiler._compiled_cache[
                name](
                frame, *inputs)
        else:
            values[name] = node.op(frame, *inputs)
    return values["crop"]
```

This execution model is structurally equivalent to TVM's `GraphExecutor`, which dispatches each operator call to either a compiled kernel or a fallback implementation depending on target support.

6 Evaluation

6.1 Experimental Setup

All measurements were collected on a MacBook Pro (M2, 8-core CPU, 16 GB unified memory) running macOS 14.5. Input frames were 640×480 RGB arrays. Each measurement reports the median of 100 iterations after 10 warmup iterations. Numba version 0.59 was used with `nopython=True` and `cache=True`.

6.2 End-to-End Pipeline Performance

Table 4 summarizes system-level targets against which CORTEX is designed. The 60% payload reduction target is conservative: center cropping alone removes 36% of pixel area, and WebP compression at quality 80 achieves 25–35% size reduction over JPEG. Combined with ROI cropping and adaptive resolution, 60–80% total reduction is achievable with existing platform APIs.

6.3 Dead-Node Elimination

Table 5 reports the reduction in per-frame latency attributable to dead-node elimination in `POWER_SAVE` mode.

In `POWER_SAVE` mode, dead-node elimination eliminates the `saliency_dft` call entirely, saving ∼0.43 ms per frame.

TABLE 5
Dead-Node Elimination: Per-Frame L2 Latency

Mode	Without DNE	With DNE
AGGRESSIVE	∼9.53 ms	∼9.53 ms (no change)
BALANCED	∼9.53 ms	∼9.53 ms (no change)
POWER_SAVE	∼9.53 ms	∼9.10 ms (−saliency_dft)

TABLE 6
Saliency DFT Latency (640×480)

Variant	Latency (ms)	Speedup	Note
Baseline	0.421	1.0×	Interpreted NumPy
Vectorized	0.225	1.9×	NumPy
JIT (Numba)	0.185	2.3×	`@jit`

6.4 JIT Compilation of Saliency DFT

Table 6 shows the per-frame latency of the saliency DFT operation before and after Numba JIT compilation.

Numerical equivalence is preserved (max diff $< 1.2\times10^{-7}$). The JIT version achieves a 2.3× speedup over the interpreted baseline.

6.5 Graph Topology Correctness

The graph visualization facility enables unit-testable topology assertions:

```
def test_power_save_dne():
    g = build_l2_graph(BatteryMode.POWER_SAVE)
    g_pruned = dead_node_elimination(g, {"
        saliency_dft": 0.0})
    assert "saliency_dft" not in g_pruned.nodes
    assert "score_fusion" in g_pruned.nodes
    assert "crop" in g_pruned.nodes
```

This test verifies that dead-node elimination correctly removes `saliency_dft` while preserving the downstream execution path.

7 Discussion

7.1 Design Rationale: Why Not TVM?

A natural question is why we chose Numba JIT over TVM, given that TVM provides more comprehensive optimization (autotuning, quantization, multi-target support). The answer is practical: CORTEX operates as a middleware layer within a mobile Python process. TVM requires model export (to Relay IR), a separate compilation step (30–60 minutes), and a runtime library deployment. For a 0.43 ms saliency kernel within a ∼9.5 ms pipeline, this overhead is unjustifiable.

Numba provides the core benefit—loop fusion, vectorization, LLVM lowering—with zero model export overhead and sub-second JIT compilation on first call. This represents the *hardware-aware optimization* principle applied pragmatically to the middleware context.

7.2 MSER as an External Call Boundary

The text detection node (`text_roi_mser`) is the pipeline's dominant bottleneck (∼8.11 ms, 85% of total) yet is marked `is_external=True` and excluded from compilation. This is intentional: MSER is implemented as a platform-native C++ routine within iOS's Vision framework and Android ML Kit. Its internal representation is inaccessible to Python-level tooling. Marking it as an external call boundary is the correct engineering decision and mirrors TVM's BYOC annotation for hardware-specific accelerators: acknowledge the boundary rather than attempt to cross it.

This boundary decision also implies that future battery savings from text detection require algorithmic changes (e.g., replacing MSER with a lighter heuristic) rather than compilation.

7.3 Graph IR as a Foundation for Future Passes

The Graph IR introduced here provides infrastructure for additional passes not yet implemented:

- *Operator fusion*: Merge `score_fusion` and `saliency_dft` into a single fused kernel, eliminating intermediate array allocation.
- *Layout transformation*: Reorder pixel layout to contiguous memory access patterns for SIMD efficiency.
- *Constant folding*: Pre-compute static components (e.g., the Gaussian center map S_c) at graph construction time.

Each of these passes has a direct analog in TVM's relay optimization pipeline, making the Graph IR a natural bridging artifact between CORTEX's Python middleware context and industrial ML compilation practice.

7.4 Limitations

The design relies on platform-specific OCR and saliency APIs, which may exhibit performance variance across iOS and Android versions. SSIM-based scene change detection may produce false negatives during gradual scene evolution. Profiling results are platform-specific and will vary across devices; MSER latency in particular depends heavily on frame content complexity. The Numba JIT compilation incurs a one-time startup cost (typically 1–3 seconds) that must be amortized across a session—acceptable for a long-running wearable application but unsuitable for single-query scenarios. Dead-node elimination currently operates only on weight-zero nodes; a more general liveness analysis over conditional request paths is left for future work.

Privacy. Camera-equipped smart glasses raise substantive privacy concerns that the research community has begun to examine [6]. CORTEX's adaptive capture engine reduces the number of frames captured per session as a direct consequence of performance optimization, which partially mitigates continuous recording concerns. Future versions will include GPS-fenced privacy zones, face-blur preprocessing, and on-device-only processing modes.

8 Conclusion

We have presented CORTEX, a hardware-agnostic middleware SDK addressing the battery–bandwidth–latency

trilemma in wearable VLM systems. Through its four-layer architecture—adaptive capture, image optimization, VLM abstraction, and context memory—CORTEX provides the missing infrastructure layer between smart glasses hardware and VLM APIs.

We further extended CORTEX with a Graph IR and compiler pipeline that bring ML compilation principles to wearable vision middleware. The Graph IR exposes the L2 processing pipeline as a manipulable DAG, enabling dead-node elimination to prevent unnecessary saliency computation in power-constrained modes. Profiling-driven target selection identifies the saliency DFT as the correct compilation target; Numba JIT compilation lowers this kernel to LLVM IR, yielding a measurable 2.3× per-frame speedup. External-call boundaries isolate platform-native operations (MSER, platform OCR) analogously to TVM's BYOC mechanism.

These additions demonstrate that compiler-style infrastructure—graph representation, lowering passes, subgraph partitioning, hardware-aware compilation—applies productively beyond traditional neural network workloads to the preprocessing pipelines of wearable AI systems. CORTEX is released under the MIT License at https://github.com/cortex-sdk.

Appendix A
Graph IR Module Structure

```
cortex/
  graph/
    __init__.py
    node.py          # Node dataclass, NodeType enum
    graph.py         # CortexGraph: add, edges,
        topo_sort
    passes/
      dead_node.py   # dead_node_elimination()
      visualize.py   # ASCII graph dump
  compiler/
    __init__.py
    compiler.py      # CortexCompiler: partition,
        compile
    kernels/
      saliency_jit.py  # Numba-compiled saliency_dft
    benchmark.py     # profiling harness
```

Appendix B
Benchmark Harness

```
import time, numpy as np
from cortex.graph import build_l2_graph
from cortex.compiler import CortexCompiler

def benchmark(node_name: str, n: int = 100) -> dict:
    frame = np.random.randint(0, 255, (480, 640, 3),
                              dtype=np.uint8)
    g = build_l2_graph()
    compiler = CortexCompiler(g)

    op = g.nodes[node_name].op
    baseline = np.median([
        (lambda t0: (op(frame), time.perf_counter()-
            t0)
         )(time.perf_counter())[1]
        for _ in range(n)]) * 1e3

    compiled_op = compiler.compile(node_name)
    compiled_op(frame)  # warmup
    compiled = np.median([
        (lambda t0: (compiled_op(frame),
                     time.perf_counter()-t0)
         )(time.perf_counter())[1]
        for _ in range(n)]) * 1e3

    return {"baseline_ms": baseline,
            "compiled_ms": compiled,
            "speedup":     baseline / compiled}
```


Acknowledgments

The author thanks the Claude AI assistant (Anthropic) for the iterative design discussions that shaped the Graph IR architecture and compiler target selection rationale documented in this paper.

References

[1] EssilorLuxottica, "Q4/Full Year 2025 Results," Press Release, Feb. 11, 2026. [Online]. Available: https://www1.essilorluxottica.com/en/newsroom/press-releases/q4-full-year-2025-results/

[2] Mentra Community, "MentraOS: Open-source smart glasses platform + SDK," GitHub, 2025. [Online]. Available: https://github.com/Mentra-Community/MentraOS

[3] Brilliant Labs, "Halo: Open source AI glasses," 2025. [Online]. Available: https://brilliant.xyz/products/halo

[4] BerriAI, "LiteLLM: Python SDK, Proxy Server to call 100+ LLM APIs in OpenAI format," GitHub, 2024. [Online]. Available: https://github.com/BerriAI/litellm

[5] Meta Platforms, "Ray-Ban Meta (Gen 2) Available Now With Up to 2x the Battery Life and Better Video Capture," Meta Newsroom, Sep. 18, 2025. [Online]. Available: https://about.fb.com/news/2025/09/ray-ban-meta-gen-2-better-battery-life-video-capture/

[6] M. Z. Iqbal and A. G. Campbell, "Adopting smart glasses responsibly: Potential benefits, ethical, and privacy concerns with Ray-Ban Stories," *AI Ethics*, vol. 3, no. 1, pp. 325–327, 2023. doi: 10.1007/s43681-022-00155-7. [Online]. Available: https://link.springer.com/article/10.1007/s43681-022-00155-7

[7] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proc. IEEE*, vol. 111, no. 1, pp. 42–91, Jan. 2023. doi: 10.1109/JPROC.2022.3226481. [Online]. Available: https://ieeexplore.ieee.org/document/9985008

[8] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, USA, Oct. 2018, pp. 578–594.

[9] C. Lattner *et al.*, "MLIR: A compiler infrastructure for the end of Moore's Law," arXiv:2002.11054 [cs.PL], Feb. 2020. [Online]. Available: https://arxiv.org/abs/2002.11054

[10] C. Leary and T. Wang, "XLA: TensorFlow, compiled," presented at the TensorFlow Dev Summit, Mountain View, CA, USA, Feb. 2017.

[11] Z. Chen *et al.*, "Bring your own codegen to deep learning compiler," arXiv:2105.03215 [cs.LG], May 2021. [Online]. Available: https://arxiv.org/abs/2105.03215

[12] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop High Performance Technical Computing in Dynamic Languages (LLVM-HPC)*, Austin, TX, USA, Nov. 2015, pp. 1–6. doi: 10.1145/2833157.2833162.

[13] X. Hou and L. Zhang, "Saliency detection: A spectral residual approach," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Minneapolis, MN, USA, Jun. 2007, pp. 1–8. doi: 10.1109/CVPR.2007.383267.

Dongchan Kim is an independent software engineer specializing in wearable AI systems and mobile middleware. He is the creator of the CORTEX open-source SDK for battery-aware wearable vision. Contact: dck.alx@gmail.com, GitHub: https://github.com/dongckim/cortex-sdk.